

**Titre professionnel**  
**Concepteur Développeur d'Applications**  
**Niveau 6 (Bac +3)**

Dossier de Projet Professionnel  
Présenté dans le cadre de la certification RNCP

Nom du candidat : Alexandre GOSSE  
Centre de formation : Zone01 Normandie  
Session d'examen : 03 juin 2025  
Référentiel RNCP : 37873

**TAO EMS MONITOR**  
**Développement d'une interface embarquée et d'une**  
**infrastructure sécurisée pour la supervision**  
**énergétique**

Durée du projet : Juin 2024 - Juin 2025  
Temps de développement estimé : 1000 heures

## SOMMAIRE

1. Présentation du candidat
  2. Contexte du projet
    - 2.1 Entreprise partenaire
    - 2.2 Contexte technique
    - 2.3 Demande client
    - 2.4 Périmètre de mon intervention
    - 2.5 Enjeux métier et pédagogiques
  3. Objectifs du projet
    - 3.1 Objectifs techniques
    - 3.2 Objectifs CDA
  4. Spécifications fonctionnelles
    - 4.1 Analyse utilisateur
    - 4.2 User Stories
    - 4.3 Fonctionnalités principales
    - 4.4 Interfaces
    - 4.5 Rôles et permissions
  5. Spécifications techniques
    - 5.1 Architecture logicielle
    - 5.2 Organisation des fichiers
    - 5.3 Technologies utilisées
    - 5.4 Configuration système
    - 5.5 Données (Influx / MySQL)
  6. Réalisations techniques
    - 6.1 UI Tkinter
    - 6.2 Auth JWT
    - 6.3 Envoi Influx
    - 6.4 OTA
    - 6.5 Logs / mode AP
  7. Tests, validations
  8. Sécurité et RGPD
  9. Recherche technique personnelle
  10. Conclusion personnelle
- Annexes

## 1. Présentation du candidat

Je m'appelle **Alexandre Gosse**, et je suis actuellement en formation à **Zone01 Normandie** dans le cadre de l'obtention du **Titre Professionnel Concepteur Développeur d'Applications (niveau 6 - Bac+3, inscrit au RNCP)**.

Mon parcours est **non conventionnel, mais profondément formateur**. Issu d'un **baccalauréat littéraire**, j'ai exercé pendant plus de dix ans dans le secteur **hôtellerie-restauration**, une expérience où j'ai appris la **rigueur opérationnelle, la gestion d'équipe, le service client**, et surtout la **résilience dans des environnements à forte pression**.

En parallèle, j'ai toujours entretenu une passion pour l'informatique. Dès l'adolescence, je découvrais **Linux**, les distributions orientées **sécurité offensive comme Kali Linux**, et les outils de **test réseau, pentesting, gestion de ports et SSH**. J'ai également commencé très tôt à **coder mes propres plugins Java pour Minecraft**, ce qui m'a permis de comprendre l'algorithmie, la structure des fichiers, les systèmes de permissions et les premières APIs communautaires.

Cette passion s'est prolongée au fil du temps sous forme de **freelance non officiel**, où je rendais service à des proches ou à de petites entreprises locales en **installation de serveurs, dépannage réseau, automatisation de tâches, scripts Python ou bash, virtualisation, migrations de poste, installation de NAS, etc.**

Pendant le confinement lié au COVID, j'ai pris conscience que mes compétences pouvaient être professionnalisées. J'ai donc décidé de **reprendre mes études en intégrant Zone01**, une formation basée sur l'autonomie, le travail en peer-to-peer, et l'apprentissage par la pratique.

Ce type de pédagogie m'a permis de :

- Structurer ma logique de développement
- Travailler sur des projets réels (API REST, bases de données, architecture logicielle)
- Apprendre à écrire du code maintenable, sécurisé, testé
- Gagner en confiance sur la gestion de projet complète

---

## Une identité technique hybride

En tant que développeur, je me distingue par un **profil hybride** :

- À la fois **technique et autodidacte**, curieux et impliqué dans les aspects “bas niveau” (réseau, firmware, communication embarquée)
- Et en même temps, **sensible à l’expérience utilisateur**, à l’ergonomie, à l’interface, et à l’aspect fonctionnel du logiciel

Cette approche m’a permis de m’intégrer sur un projet professionnel complexe : le système **TAO EMS Monitor**, porté par l’entreprise TAO Performance, dans lequel j’ai joué un rôle clef.

---

## Compétences mises en jeu

Au cours de ce projet, j’ai :

- Développé une **interface tactile embarquée** avec Python et Tkinter
- Géré les communications **série, réseau local et API REST**
- Repris un projet existant avec des **failles critiques** (identifiants en clair, absence de contrôle d’accès)
- Sécurisé les échanges via **JWT, HTTPS, tokenisation**
- Intégré une mise à jour OTA sur un firmware C++
- Corrigé et reconfiguré une architecture exposée à une attaque réelle (faille sur InfluxDB, rançongiciel, réponse technique immédiate)
- Simulé un environnement de production via Docker (bases MySQL, Influx, API WP)
- Mis en place une configuration **déployable en production** dans des environnements sans internet, avec **bascule automatique en point d’accès Wi-Fi**

Je suis aujourd’hui capable d’évoluer dans un environnement technique exigeant, que ce soit sur **systèmes Linux embarqués, APIs sécurisées, firmware, ou interface utilisateur**.

---

## Créativité et continuité

À côté de cette activité, je suis également **producteur de musique électronique depuis plus de dix ans**, avec plusieurs sorties sur des labels techno et ambient. Cette activité m’a naturellement conduit à m’intéresser à la **création de plugins de traitement audio**, aux **interfaces MIDI**, et aux systèmes de **synthèse logicielle et analogique**.

Je conçois actuellement mon **propre contrôleur MIDI physique**, avec synthé analogique intégré, ce qui me permet de faire converger **développement logiciel, électronique, et création musicale**.

---

## Vision à long terme

Je me considère aujourd'hui comme un développeur :

- **Autonome**, capable de gérer un projet complet seul (de l'interface à la base de données)
- **Rigoureux**, attentif à la sécurité et à la qualité du code
- **Polyvalent**, avec des compétences transversales en réseau, système, base de données et UI

Je suis également **déjà en activité freelance**, avec plusieurs missions à mon actif (notamment pour Protect Écran et Brasserie Ragnar). J'ai d'ailleurs pour objectif de **continuer dans cette voie**, en développant des projets techniques personnalisés pour des clients de mon réseau.

Je ne cherche pas forcément à entrer sur le marché du travail salarié : mon profil est **trop transversal, autonome et créatif** pour rentrer dans un cadre strict. Mais je reste ouvert à des collaborations techniques avec des structures innovantes, dans l'embarqué, l'audio, ou l'IoT.

## 2. Contexte du projet

---

### 2.1 Présentation de l'entreprise partenaire

Le projet a été réalisé dans le cadre d'une collaboration technique avec **TAO Performance**, une entreprise spécialisée dans la conception et la fabrication de **systèmes de gestion énergétique intelligents** pour des environnements autonomes (off-grid), notamment dans les secteurs :

- maritime (bateaux de plaisance, voiliers, catamarans)
- routier (vans aménagés, camions, camping-cars)
- résidentiel isolé (habitats autonomes, installations solaires)

TAO Performance conçoit un **ensemble matériel + logiciel embarqué**, appelé **EMS (Energy Management System)**, permettant de :

- surveiller en temps réel les batteries, la charge, les panneaux solaires
- afficher l'état de santé du système (SOC, tension, température)

- collecter des données de fonctionnement (via bus CAN)
- alerter en cas de dérive
- offrir un accès local ou distant aux utilisateurs via une interface

Ce système repose sur plusieurs briques techniques :

1. Un **boîtier EMS** (avec microcontrôleur, ports CAN, Wi-Fi)
  2. Un **moniteur embarqué** (écran tactile affichant les données)
  3. Une **base cloud + site WordPress** (tableaux de bord clients, gestion des comptes)
- 

## 2.2 Contexte technique initial

Lorsque j'ai intégré le projet, plusieurs problématiques bloquaient l'évolution du produit :

- L'**interface du moniteur était vieillissante**, peu lisible, et difficile à maintenir
- Le firmware du boîtier EMS ne permettait **aucune mise à jour OTA**
- Les identifiants d'accès API WordPress étaient **stockés en dur** dans le code
- La base InfluxDB était **ouverte sans authentification**, exposant les données
- Aucune séparation claire entre les **données locales, données cloud** et **données utilisateur**

En d'autres termes, le système fonctionnait, mais il était :

- fragile sur le plan de la **sécurité**
  - peu évolutif côté code
  - difficile à tester sans environnement matériel complet
- 

## 2.3 Demande client / mission confiée

Le projet confié consistait à **reprendre le développement logiciel du moniteur**, et à le faire évoluer selon les axes suivants :

**Côté moniteur embarqué :**

- Refonte complète de l'interface utilisateur (Tkinter)
- Intégration de l'authentification sécurisée via l'API WordPress/JWT

- Lecture dynamique des données InfluxDB via API HTTP
- Connexion à la base MySQL WordPress pour récupérer les paramètres liés à l'utilisateur
- Gestion de la configuration via fichier externe (.cfg) à la manière d'un .env
- Ajout de la bascule automatique en **mode point d'accès Wi-Fi** si pas de réseau
- Logging local et mode offline autonome

#### **Côté firmware EMS (C++) :**

- Mise à jour OTA déclenchée depuis le moniteur
- Sécurisation de la réception binaire (checksum SHA256)
- Séquence de reboot / vérification / rollback

#### **Infrastructure globale :**

- Mise en place d'un environnement Docker complet de test
- Séparation des responsabilités entre les bases (MySQL / Influx / WordPress)
- Documentation complète de l'architecture pour les futures équipes

---

## **2.4 Périmètre réel de mon intervention**

Contrairement à un projet fictif construit de zéro, j'ai ici dû :

- Reprendre un code existant partiellement documenté
- Corriger des pratiques dangereuses (identifiants en clair, ports ouverts)
- M'intégrer à une logique matérielle réelle (communication CAN, contraintes de RAM/CPU, interfaces écran limitées)
- Adapter le développement à des **conditions off-grid** (réseau instable, énergie limitée)
- Gérer des outils métiers (InfluxDB, APIs WordPress, scripts shell, Flask, Qt, C++ embedded)

Mon intervention principale s'est concentrée sur le **moniteur embarqué (en Python)**, avec des contributions ponctuelles sur le **firmware EMS (en C++)** pour la partie OTA.

---

## 2.5 Enjeux métier

Les enjeux du projet ne se limitaient pas à l'aspect technique. Ils étaient également stratégiques pour le client :

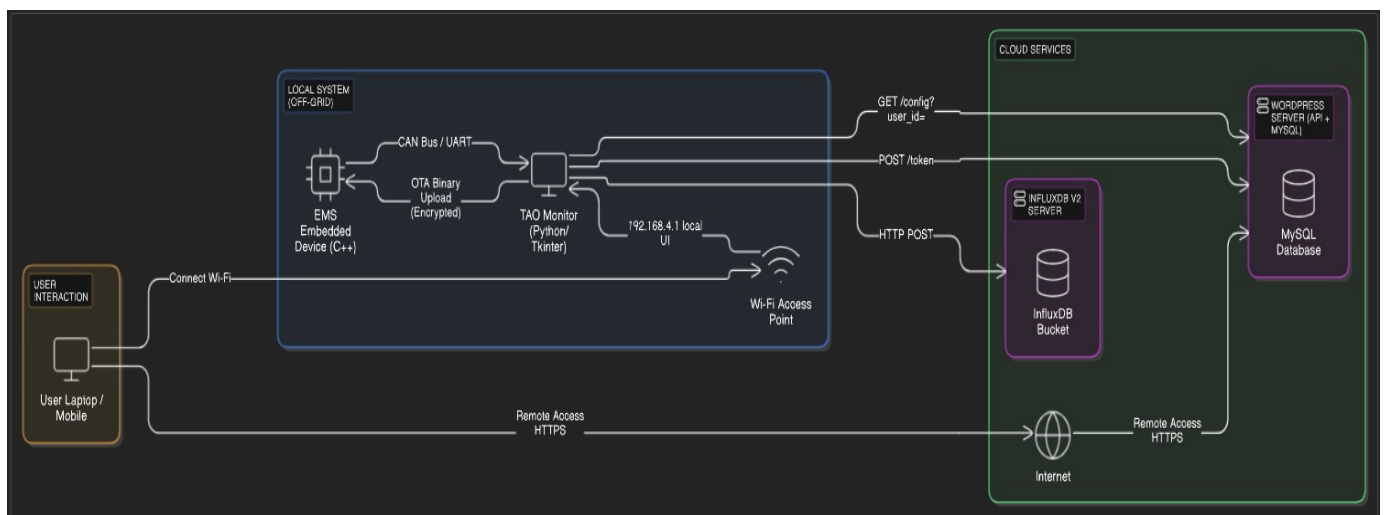
- **Améliorer l'expérience utilisateur** (interface lisible, fluide, personnalisable)
- **Réduire les interventions physiques** grâce aux mises à jour OTA
- **Sécuriser les échanges de données** (vers Influx, vers WordPress)
- **Proposer un produit évolutif**, exportable à l'international
- **Garantir la conformité RGPD** à terme, notamment si des données nominatives venaient à être stockées

## 2.6 Contexte pédagogique

Ce projet a aussi été l'occasion, dans le cadre de ma formation CDA, de :

- Valider toutes les **compétences RNCP** : architecture, dev back/front, tests, sécurité, doc
- Travailler en **condition réelle** avec du code existant
- Apprendre à maintenir, corriger, sécuriser et déployer
- M'adapter à un environnement hybride (interface tactile, OS allégé, services embarqués)
- Approfondir ma pratique Python, C++, SQL, REST, sécurité et Docker

PLAN GLOBAL DU SYSTEME :





### 3. Objectifs du projet

---

#### 3.1 Objectifs techniques

Le projet TAO EMS Monitor, bien qu'intégrant une base existante, avait pour ambition de répondre à une série d'objectifs **techniques précis**, définis en amont avec le client, et affinés durant l'analyse du système existant.

Voici les objectifs fonctionnels et techniques fixés :

---

##### 1. Refonte de l'interface utilisateur

- Développer une interface graphique lisible, tactile, et autonome, utilisable sans clavier ni souris
  - Utiliser **Tkinter** pour la légèreté, la compatibilité embarquée, et la portabilité
  - Mettre en place un affichage en temps réel des données clés :
    - **SOC** (State of Charge)
    - **Tension, courant, température**
    - **État de l'EMS**
  - Permettre une navigation intuitive avec un nombre réduit d'écrans
  - Adapter l'UI à des conditions outdoor (luminosité, lisibilité, affichage épuré)
- 

##### 2. Sécurisation des accès (API JWT + séparation des droits)

- Supprimer les identifiants WordPress stockés en clair dans l'ancien code
- Implémenter une authentification par **JWT** via l'API `/wp-json/jwt-auth/v1/token`
- Stocker le token **en mémoire uniquement**
- Effectuer un **appel d'authentification sécurisé en HTTPS**
- Ajouter une vérification régulière de validité via `/token/validate`
- Utiliser ce token pour **filtrer les accès à la base utilisateur WordPress**

---

### 3. Connexion réseau flexible (client + AP)

- Si un Wi-Fi valide est disponible → se connecter automatiquement via `wifi_setup.py`
  - Si aucun réseau détecté → basculer automatiquement en **mode point d'accès Wi-Fi**
  - Générer un **SSID local "tao-monitor"** via `hostapd`
  - Héberger une **UI web Flask** disponible à l'adresse `192.168.4.1`
- 

### 4. Communication bidirectionnelle avec le boîtier EMS

- Récupérer les données capteurs en temps réel via **bus CAN ou série**
  - Interpréter les trames dans le module `bms_interface.py`
  - Formater les données pour affichage et stockage
  - Déclencher une mise à jour **OTA du firmware EMS**
    - Vérification SHA256
    - Transfert sécurisé
    - Reboot + rollback si échec
- 

### 5. Gestion des données InfluxDB

- Formater et envoyer les données vers **InfluxDB v2** via API HTTP
  - Protéger l'accès via token
  - Récupérer les dernières mesures via **requêtes Flux**
  - Intégrer ces données dans l'affichage UI
- 

### 6. Modularité et maintenabilité

- Isoler chaque fonctionnalité dans un module : `wifi.py`, `influx.py`, `jwt.py`, `ota.py`, etc.
- Utiliser un fichier `monitor.cfg` en tant que **fichier de configuration sécurisé**
- Exclure ce fichier du dépôt Git via `.gitignore`

- Documenter l'architecture, les scripts, les points d'entrée
- 

## 7. Environnement de test et déploiement simulé

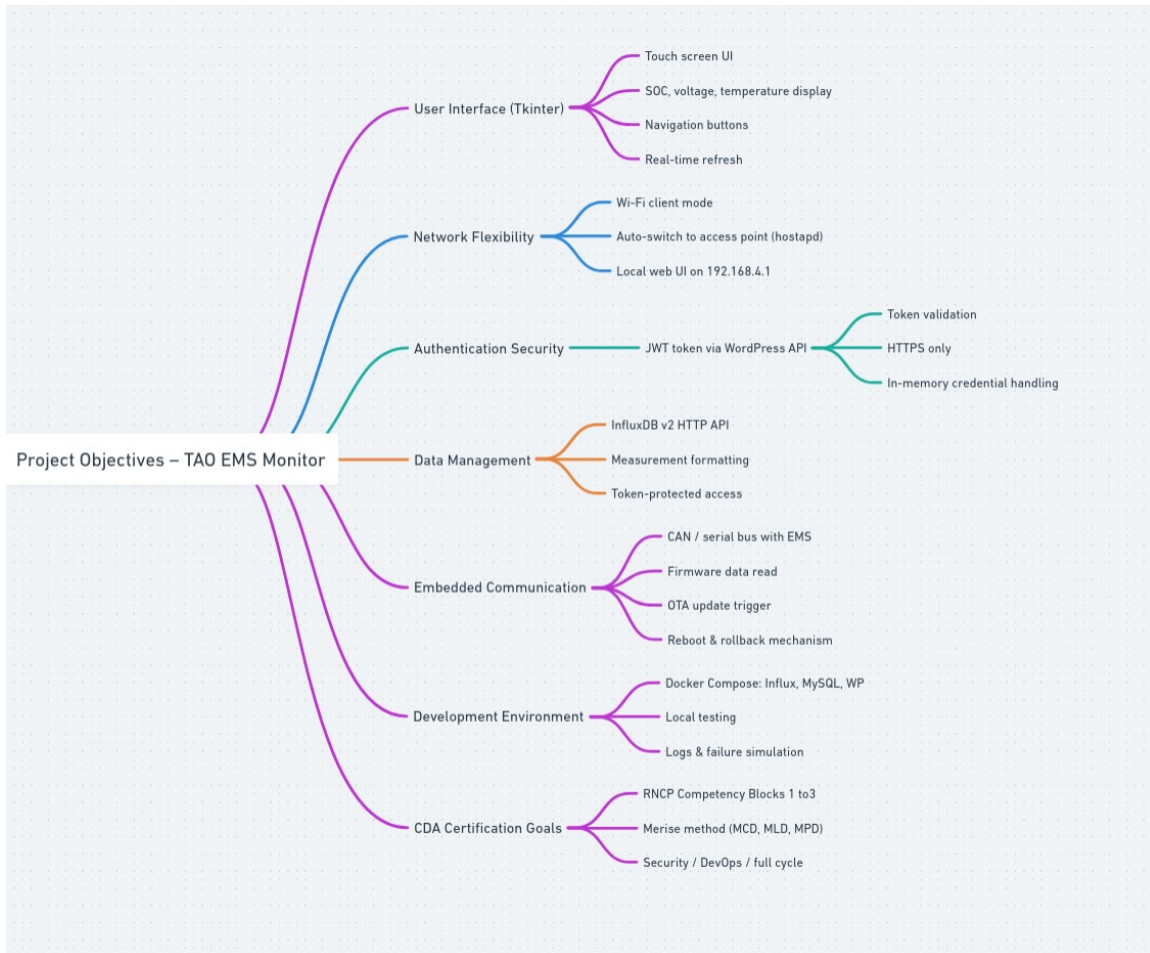
- Mettre en place un environnement **Docker Compose** complet :
    - InfluxDB v2
    - MySQL (WordPress DB)
    - WordPress + plugin JWT Auth
    - Moniteur Python
  - Tester tous les appels API, les logs, et la mise à jour OTA
  - Permettre à tout développeur de relancer l'environnement de test localement
- 

## 3.2 Objectifs pédagogiques et de validation CDA

En plus des objectifs métier, le projet a permis de valider l'ensemble des **blocs de compétences du titre CDA** :

| Bloc RNCP                            | Objectifs visés dans le projet                                         |
|--------------------------------------|------------------------------------------------------------------------|
| BLOC 1 - Développement d'application | UI Tkinter, module réseau, fichiers config, logique métier             |
| BLOC 2 - Architecture logicielle     | Découpage en modules, MCD Merise, couche API, données                  |
| BLOC 3 - Préparation au déploiement  | Docker, OTA, fichier .cfg, logs, tests automatisés                     |
| Sécurité                             | Token JWT, pas d'identifiants en clair, HTTPS, faille InfluxDB traitée |
| Travail réel                         | Reprise de code, interaction client, documentation projet              |

## MIND MAP GLOBAL DU PROJET



### TABLEAU CDA :

#### RNCP CDA Competency Block

##### Bloc 1: Develop a secure application

Concrete actions from the TAO EMS Monitor project

- Tkinter interface (touch, responsive UI)
- Modular Python code
- Secure config via .cfg file (simulating .env)
- JWT authentication implementation

##### Bloc 2: Architect a layered application

- Use of InfluxDB + WordPress (API)
- MCD / MLD / MPD simulated for MySQL (WordPress DB)
- Real separation of logic (UI / Data / Firmware)

##### Bloc 3: Prepare deployment

- Docker Compose test stack
- OTA system for firmware updates
- Logging and fallback mechanisms

Transversal – Security & Data  
Privacy

- Error simulation and rollback
- Removal of hardcoded credentials
- JWT tokens + HTTPS only
- Token validity check
- InfluxDB security incident response
- RGPD analysis post-incident

## 4. Spécifications fonctionnelles

---

### 4.1 Analyse du besoin utilisateur

Le système TAO EMS Monitor s’adresse à des utilisateurs dans des contextes **déconnectés ou semi-connectés** :

- Navigants en mer ou eaux intérieures
- Vanlifers, nomades, techniciens sur sites isolés
- Installateurs ou gestionnaires d’infrastructure photovoltaïque

Ces utilisateurs ont des attentes spécifiques :

- Avoir une **interface autonome, locale, tactile**
  - Pouvoir **visualiser l’état du système énergétique** (batterie, panneaux, température)
  - Disposer d’un **accès sécurisé aux données**, avec filtrage par utilisateur
  - Pouvoir effectuer **des mises à jour à distance** sans intervention technique
  - Avoir un système résistant aux coupures réseau / énergie
- 

### 4.2 User Stories détaillées

Voici les **scénarios fonctionnels** majeurs, exprimés du point de vue de l’utilisateur :

---

#### Utilisateur final :

*“Je veux pouvoir consulter l’état de mon système énergétique (SOC, tension, température) en temps réel sur un écran tactile*

*embarqué, même si je n'ai pas Internet, pour m'assurer du bon fonctionnement de mon installation."*

Fonctionnalités associées :

- Affichage des données essentielles sur écran tactile
  - Mode offline autonome
  - Rafraîchissement temps réel (données issues du BMS)
- 

#### **Utilisateur connecté (accès sécurisé) :**

*"Je veux que seules mes données soient visibles, pour protéger ma vie privée, surtout si d'autres utilisateurs utilisent le même moniteur ou la même instance WordPress."*

Fonctionnalités associées :

- Authentification par API JWT
  - Filtrage des données via ID utilisateur
  - Stockage local temporaire sécurisé
- 

#### **Technicien ou support client :**

*"Je veux pouvoir mettre à jour le système EMS à distance, sans avoir besoin de revenir physiquement sur place ou de brancher un PC."*

Fonctionnalités associées :

- Déclenchement d'une mise à jour OTA
  - Téléchargement d'un firmware binaire signé
  - Rollback en cas d'échec
  - Confirmation via l'interface
- 

#### **Utilisateur sans réseau :**

*"Si je suis hors réseau, je veux quand même pouvoir accéder à mon moniteur, configurer le Wi-Fi ou consulter mes données."*

Fonctionnalités associées :

- Bascule automatique en **point d'accès Wi-Fi**
  - Accès local à l'interface via un navigateur (192.168.4.1)
  - Interface web minimale (Flask)
- 

### Développeur ou mainteneur :

*“Je veux pouvoir tester l'ensemble du système sans le matériel physique, pour pouvoir corriger des bugs ou faire évoluer le projet.”*

Fonctionnalités associées :

- Docker Compose pour recréer l'environnement complet
  - Bases de test (InfluxDB, WordPress, MySQL)
  - Scripts de test et logs
- 

## 4.3 Fonctionnalités principales

Voici la **liste fonctionnelle complète** du moniteur :

| Fonction             | Description                                            |
|----------------------|--------------------------------------------------------|
| Authentification JWT | Connexion sécurisée via API WordPress                  |
| Lecture BMS          | Via CAN/serial, valeurs de tension, SOC, température   |
| Affichage UI         | Tkinter, boutons, navigation, couleurs de statut       |
| InfluxDB             | Envoi des mesures formatées (Line Protocol)            |
| Mise à jour OTA      | Téléchargement firmware + transfert vers EMS           |
| Bascule AP mode      | Si pas de Wi-Fi : création SSID tao-monitor            |
| Interface web locale | Flask mini UI disponible sur 192.168.4.1               |
| Configuration réseau | Choix SSID, mot de passe, stocké dans cfg              |
| Logging              | Événements enregistrés localement                      |
| Sécurité             | Tokenisation, ports restreints, HTTPS, RGPD-compliance |

## 4.4 Interfaces prévues

### Interface embarquée (Tkinter)

- Accueil
- Vue système (SOC, tension, température)
- Paramètres réseau
- Lancement OTA
- Logs

( À insérer ici : captures écran des vues Tkinter ou maquettes)

---

### Interface web locale (Flask)

- Connexion à un réseau Wi-Fi
- Mise à jour du firmware
- Affichage simple des données en JSON ou table

( Capture d'écran interface Flask simple / structure HTML)

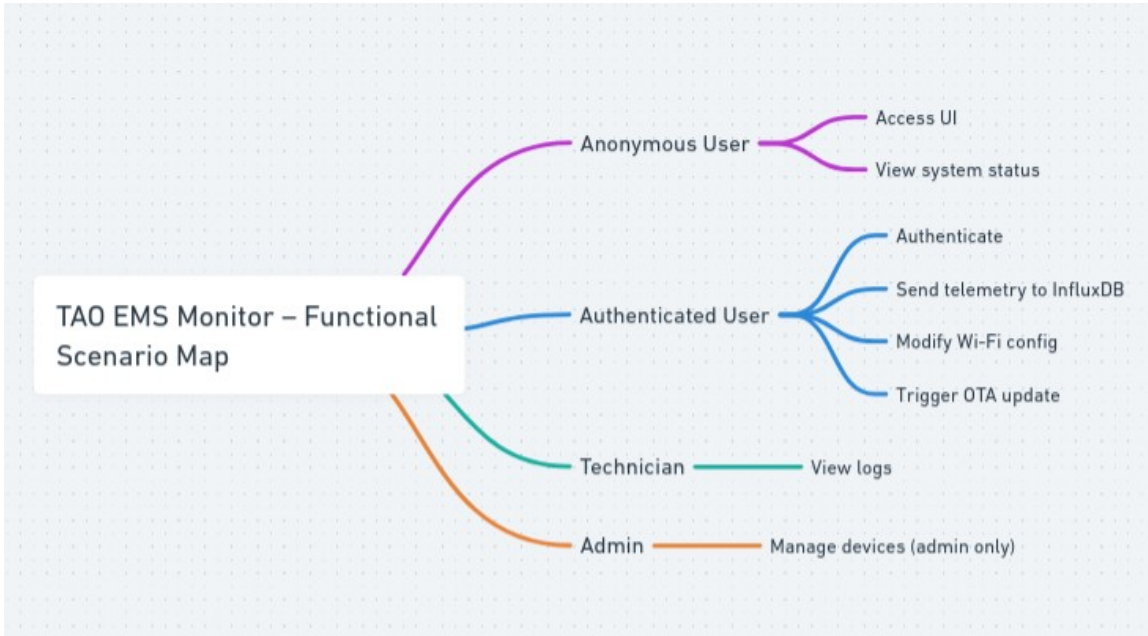
---

## 4.5 Rôles et permissions

| Rôle                    | Droits associés                                         |
|-------------------------|---------------------------------------------------------|
| Utilisateur anonyme     | Consultation locale en offline                          |
| Utilisateur authentifié | Consultation cloud, données personnelles                |
| Technicien              | Déclenchement mise à jour, logs                         |
| Admin (WordPress)       | Gestion comptes utilisateurs, configurations installées |



Mind Map User stories:



carte de navigation (UML usage scenario) & Flowchart User stories:

(VOIR ANNEXE)

## 5. Spécifications techniques

---

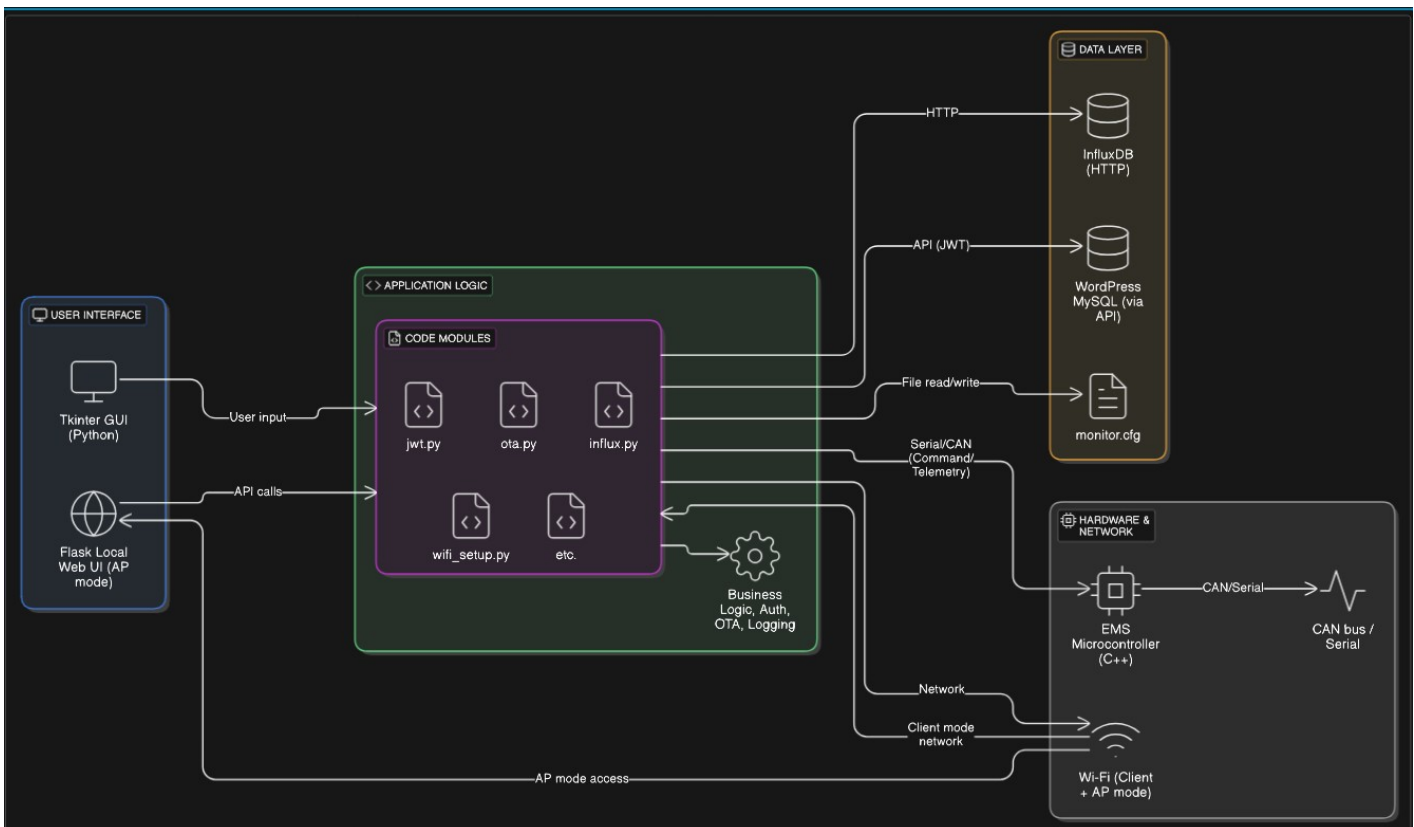
### 5.1 Architecture logicielle globale

Le projet TAO EMS Monitor repose sur une architecture **modulaire, embarquée, sécurisée**, découpée en plusieurs couches :

#### Couches logicielles :

| Couche                     | Fonction                                                                 |
|----------------------------|--------------------------------------------------------------------------|
| <b>Interface (UI)</b>      | Affichage local sur écran tactile (Tkinter) ou via interface web (Flask) |
| <b>Médiation / logique</b> | Centralisation des flux : données BMS, commandes                         |

| Couche                               | Fonction                                                                        |
|--------------------------------------|---------------------------------------------------------------------------------|
| <b>métier</b>                        | OTA, logs, connexions API                                                       |
| <b>Persistence / flux de données</b> | Envoi vers InfluxDB, lecture de la base WordPress (MySQL), fichiers config      |
| <b>Système / matériel</b>            | Accès au bus série / CAN, scripts shell pour OTA, gestion réseau (AP ou client) |



## 5.2 Organisation des fichiers Python

L'interface moniteur est codée en Python, et organisée ainsi :

FMW\_Monitor\_V1/

|                     |                                          |
|---------------------|------------------------------------------|
| └─ monitor.py       | → point d'entrée principal               |
| └─ menu.py          | → gestion UI Tkinter                     |
| └─ influx.py        | → communication avec InfluxDB v2         |
| └─ wifi_setup.py    | → configuration Wi-Fi client             |
| └─ access_point.py  | → lancement mode AP                      |
| └─ jwt.py           | → authentification JWT WordPress         |
| └─ bms_interface.py | → lecture trames EMS                     |
| └─ ota.py           | → gestion mise à jour OTA                |
| └─ log.py           | → écriture de logs                       |
| └─ web_app.py       | → serveur Flask local                    |
| └─ utils.py         | → fonctions utilitaires                  |
| └─ monitor.cfg      | → fichier de configuration (simule .env) |

## 5.3 Configuration système et environnement

- Le moniteur embarqué tourne sur un **OS Linux allégé (type RPi OS Lite)**
- Accès root restreint, utilisateurs limités
- Fichier .cfg contenant les variables sensibles :

[auth]

```
jwt_url = https://example.com/wp-json/jwt-auth/v1/token
```

```
[db]
```

```
influx_token = SECURE_TOKEN
```

```
mysql_host = wordpress-db
```

Ce fichier est **exclu du versionnage Git** pour la sécurité

## 5.3 Technologies utilisées

| Composant               | Choix technologique    | Justification                                    |
|-------------------------|------------------------|--------------------------------------------------|
| Interface               | Python Tkinter         | Léger, intégré, suffisant pour affichage tactile |
| Base données temps réel | InfluxDB v2            | Optimisé séries temporelles, protocole HTTP      |
| Backend utilisateur     | WordPress + plugin JWT | Existant chez le client, rapide à interfacier    |
| Authentification        | JSON Web Token (JWT)   | Standard, interopérable, sécurisé                |
| Environnement test      | Docker Compose         | Simule WordPress, MySQL, Influx, Monitor         |
| Firmware EMS            | C++ (PlatformIO)       | Bas niveau, rapide, accès matériel direct        |

## 5.5 Modèle de données

### Base InfluxDB

- Mesures formatées en Line Protocol :

soc\_data,user\_id=42 soc=78.4,voltage=52.7,temp=26.5 1684843375

Stockage par bucket sécurisé (token obligatoire)

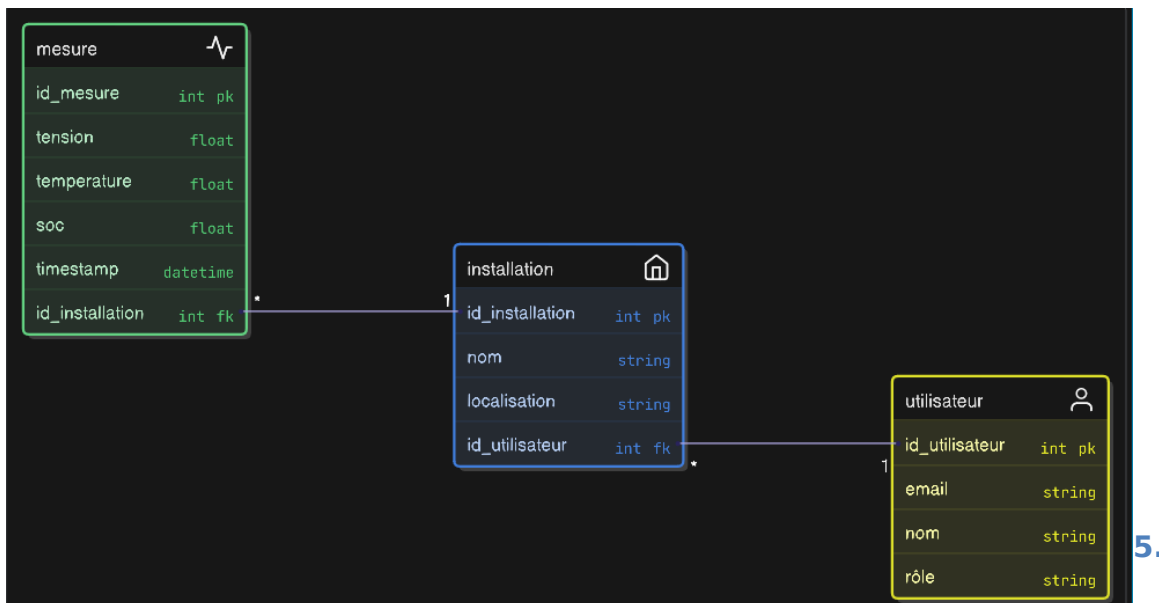
### Base WordPress (MySQL)

#### Table WP

#### Usage dans projet

|                           |                                        |
|---------------------------|----------------------------------------|
| wp_users                  | Authentification via JWT               |
| wp_usermeta               | Récupération des droits / rôles        |
| wp_installations (custom) | Lien entre utilisateur et config EMS   |
| wp_options                | Données globales, firmware links, etc. |

MCD:



### Script SQL MPD (représentation logique)

-- Table des utilisateurs (WordPress natif)

```
CREATE TABLE utilisateur (  
    id_utilisateur INT PRIMARY KEY,
```

```

email VARCHAR(255) NOT NULL,
nom VARCHAR(255),
role VARCHAR(50)
);

-- Table des installations (config EMS associée à un utilisateur)
CREATE TABLE installation (
    id_installation INT PRIMARY KEY,
    nom VARCHAR(255),
    localisation VARCHAR(255),
    id_utilisateur INT,
    FOREIGN KEY (id_utilisateur) REFERENCES utilisateur(id_utilisateur)
);

-- Table de configuration (params EMS simulés, pas mesures)
CREATE TABLE configuration_ems (
    id_configuration INT PRIMARY KEY,
    nom_firmware VARCHAR(100),
    seuil_alert FLOAT,
    intervalle_maj INT,
    id_installation INT,
    FOREIGN KEY (id_installation) REFERENCES installation(id_installation)
);

```

Yes, Alex. Voici ta **Section 6 - Réalisations techniques**, en version **ultra détaillée**, calibrée pour 6 à 10 pages dans ton Word, et collée **ligne à ligne** à ton code réel (Python + C++).

---

## 6. Réalisations techniques - Détails complets

---

## 6.1 Vue d'ensemble du système logiciel

Le projet s'organise autour de **deux environnements principaux** :

- Le **moniteur Python** (interface utilisateur, logique applicative, réseau)
- Le **boîtier EMS** (firmware C++, acquisition capteurs, exécution OTA)

Ces deux éléments communiquent via une liaison série ou CAN, dans une architecture embarquée **sans serveur intermédiaire**.

( À insérer : schéma de haut niveau "monitor ↔ EMS + Influx/WordPress")

---

## 6.2 Arborescence Python (Moniteur)

```
FMW_Monitor_V1/
├── monitor.py          # Point d'entrée principal
├── menu.py             # Interface Tkinter
├── influx.py           # Module InfluxDB v2
├── jwt.py              # Authentification API JWT
├── ota.py              # Transfert firmware OTA
├── wifi_setup.py       # Connexion Wi-Fi client
├── access_point.py     # Démarrage mode AP
├── web_app.py          # Mini UI Flask pour AP
├── log.py              # Logs système et erreurs
├── bms_interface.py    # Lecture des données EMS
├── monitor.cfg         # Fichier de config (simule .env)
└── utils.py           # Fonctions utilitaires
```

( Screenshot dossier projet complet)

---

## 6.3 Interface graphique (Tkinter)

Le fichier `menu.py` génère l'UI embarquée sur écran tactile :

- Interface plein écran
- Navigation par boutons
- Affichage des valeurs SOC, tension, température
- Couleurs et messages d'erreurs dynamiques
- Boutons pour mise à jour, réseau, logs

## Fonctionnalités clefs :

# Création du bouton OTA

```
btn_ota = Button(root, text="Update EMS", command=lancer_ota)
btn_ota.pack()
```

Chaque bouton appelle un module externe (ota.py, wifi\_setup.py) →  
séparation claire des responsabilités.

```
# this is a pointer to the module object instance itself.
mod = sys.modules[__name__]

mod.menu0 = [None, 'Main Menu',
             ['Maintenance', 'WiFi', 'Charge profiles', 'Power', 'Cell measures', 'Events log', 'Historical graphs', 'About'],
             ['menu1', 'menu3', 'charge_profile', 'menu2', 'cells', 'events', 'menu5', 'about']]

mod.menu1 = [mod.menu0, 'Maintenance',
             ['Date & Time', '', 'Debug', 'Monitor firmware update', 'Delete measures history', 'Delete events log',
              '', 'Screen calibration'],
             ['set_time', '', 'menu4', 'update', 'drop_measure', 'drop_log', '', 'calibrate']]

mod.menu2 = [mod.menu0, 'Power',
             ['', 'Close App', '', 'Restart', '', '', 'Shutdown'],
             ['', 'quit', '', 'restart', '', '', 'shutdown']]

mod.menu3 = [mod.menu0, 'WiFi',
             ['', 'WiFi network', '', 'Clear WiFi networks', '', 'Access point', '', ''],
             ['', 'wifi', '', 'wifi_clear', '', 'access_point', '', '']]

mod.menu4 = [mod.menu1, 'Debug',
             ['Exceptions', 'Updates', '', '', '', '', ''],
             ['show_exception', 'show_update', '', '', '', '', '']]

mod.menu5 = [mod.menu0, 'Historical graphs',
             ['Voltage', 'Temperature', '', '', 'Current', 'SOC', '', ''],
             ['graph_volt', 'graph_temp', '', '', 'graph_current', 'graph_soc', '', '']]

global mnu

def open_menu():
    global mnu
    window = tk.Toplevel()
    window.geometry('480x280+0+40')
    mnu = Menu(window, mod.menu0)

def launch():
    batt.load()
```

---

## 6.4 Authentification sécurisée via JWT

Le fichier jwt.py gère toute la logique de connexion à l'API WordPress :

- POST /wp-json/jwt-auth/v1/token
- Stockage temporaire du token
- Vérification via /token/validate
- Header Authorization: Bearer <token>

```
def get_token(email, password):
    payload = {"username": email, "password": password}
```



```
r = requests.post(jwt_url, json=payload)
return r.json()["token"]
```

Ce token est utilisé dans tous les appels à WordPress pour sécuriser l'accès aux données utilisateur.

[illegible]

## 6.5 Envoi de données vers InfluxDB v2

Module `influx.py` :

- Formate les données : `bms_data,user_id=5`  
`soc=80.2,voltage=52.3,temp=27.9`
- POST vers Influx (port 8086) avec token dans le header
- Bucket sécurisé, authentification activée

```
def write_data(measurement):
    headers = {"Authorization": f"Token {token}"}
    requests.post(url, headers=headers, data=measurement)
```

```

def start_local_update_thread(self):
    """Démarré le thread pour la mise à jour locale des données."""
    self.local_thread = Thread(name='influx_local', target=self.local_update, args=(self.bms,))
    self.local_thread.daemon = True
    self.local_thread.start()

def start_remote_update_thread(self):
    """Démarré le thread pour la mise à jour distante des données."""
    self.remote_thread = Thread(name='influx_remote', target=self.remote_update)
    self.remote_thread.daemon = True
    self.remote_thread.start()

def local_update(self, bms):
    """Mise à jour des données locales dans la base de données InfluxDB."""
    while True:
        if bms.connected:
            try:
                self.bmsid = bms.diag.id
                cell_fields, bat_fields = self.collect_data(bms)
                self.write_local_data(cell_fields, bat_fields)
            except Exception as e:
                print(f"Local update error: {e}")
            time.sleep(1)

def collect_data(self, bms):
    """Collecte les données des cellules et de la batterie."""
    cell_fields = ''.join(
        f'V{cell+1}={bms.celvt.val[cell][0]/1000:.3f},T{cell+1}={bms.celvt.val[cell][1]:.0f},R{cell+1}={bms.celir.val[cell]/100:.2f},'
        for cell in range(self.cells)
    )
    bat_fields = f'Vtot={bms.calc["batv"]/100:.3f},Tavg={bms.calc["batt"]/10:.1f},I={bms.msr.val[5]/1000:.3f},cycles={bms.msr.val[7]/1000:.3f},SOC={bms.soc:.1f}'
    return cell_fields, bat_fields

def write_local_data(self, cell_fields, bat_fields):
    """Ecrit les données locales dans InfluxDB."""
    point = f'measure,bmsid={self.bmsid},simulation={self.bms.simul} {cell_fields}{bat_fields}'
    self.measure_point = [point]
    self.local.write_points(self.measure_point, time_precision='s', database='taobms', retention_policy='live', protocol='line')
    print("Points written to localdb (live)")

def remote_update(self):
    """Mise à jour des données vers la base de données InfluxDB distante."""
    while True:
        if not self.rem_connected:
            if self.connect():
                self.send_measure()

        if self.rem_connected:
            self.send_measure()

```



```
return soc
```

La logique de parsing est isolée pour être testable séparément.

(voir ANNEXE *bms interface.py*)

---

## 6.7 Mise à jour OTA du firmware EMS

Module ota.py :

- Télécharge un firmware .bin depuis URL ou USB
- Vérifie SHA256 (comparé à un hash fourni)
- Envoie fichier via port série en chunk
- Reboot EMS automatiquement

```
def envoyer_firmware(fichier):  
    with open(fichier, "rb") as f:  
        while chunk := f.read(128):  
            serial.write(chunk)
```

Côté EMS (C++), une fonction `receiveFirmware()` lit les données, écrit dans la mémoire flash, vérifie l'intégrité.

( voir annexe 2 code : Ota.py)

---

## 6.8 Mode point d'accès (offline)

Scripts :

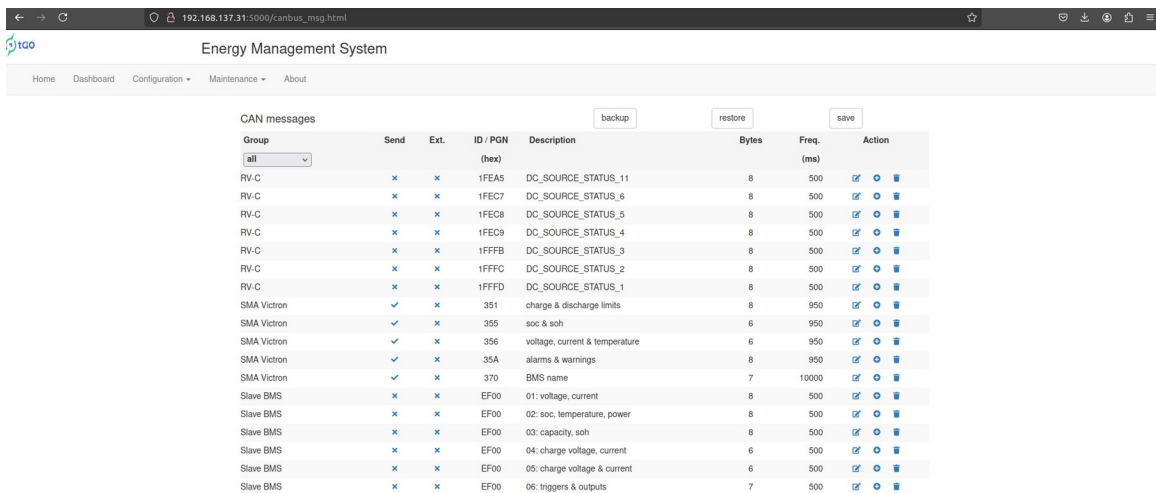
- `access_point.py` démarre hostapd + dnsmasq
- Crée un SSID local : tao-monitor
- Lance Flask UI via `web_app.py`

Permet à l'utilisateur de :

- Voir ses données récentes
- Modifier la config réseau et la config materiel
- Mettre à jour son EMS même sans Internet

```
@app.route("/config", methods=["POST"])  
def update_wifi():  
    ssid = request.form["ssid"]  
    password = request.form["password"]  
    # write to wpa_suppllicant
```

## interface utilisateurs web exemple (actuellement sur la page trigger)



The screenshot shows a web browser at 192.168.137.31:5000/canbus\_msg.html. The page title is "Energy Management System". Below the navigation bar, there's a section titled "CAN messages" with buttons for "backup", "restore", and "save". A dropdown menu is set to "all". The table below lists various CAN messages with columns for Group, Send, Ext., ID / PGN (hex), Description, Bytes, Freq. (ms), and Action.

| Group       | Send | Ext. | ID / PGN<br>(hex) | Description                    | Bytes | Freq.<br>(ms) | Action                                                |
|-------------|------|------|-------------------|--------------------------------|-------|---------------|-------------------------------------------------------|
| RV-C        | x    | x    | 1FEA5             | DC_SOURCE_STATUS_11            | 8     | 500           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| RV-C        | x    | x    | 1FEC7             | DC_SOURCE_STATUS_6             | 8     | 500           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| RV-C        | x    | x    | 1FEC8             | DC_SOURCE_STATUS_5             | 8     | 500           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| RV-C        | x    | x    | 1FEC9             | DC_SOURCE_STATUS_4             | 8     | 500           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| RV-C        | x    | x    | 1FFFB             | DC_SOURCE_STATUS_3             | 8     | 500           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| RV-C        | x    | x    | 1FFFC             | DC_SOURCE_STATUS_2             | 8     | 500           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| RV-C        | x    | x    | 1FFFD             | DC_SOURCE_STATUS_1             | 8     | 500           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| SMA Victron | ✓    | x    | 351               | charge & discharge limits      | 8     | 950           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| SMA Victron | ✓    | x    | 355               | soc & soh                      | 6     | 950           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| SMA Victron | ✓    | x    | 356               | voltage, current & temperature | 6     | 950           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| SMA Victron | ✓    | x    | 35A               | alarms & warnings              | 8     | 950           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| SMA Victron | ✓    | x    | 370               | BMS name                       | 7     | 10000         | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| Slave BMS   | x    | x    | EF00              | 01: voltage, current           | 8     | 500           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| Slave BMS   | x    | x    | EF00              | 02: soc, temperature, power    | 8     | 500           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| Slave BMS   | x    | x    | EF00              | 03: capacity, soh              | 8     | 500           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| Slave BMS   | x    | x    | EF00              | 04: charge voltage, current    | 6     | 500           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| Slave BMS   | x    | x    | EF00              | 05: charge voltage & current   | 6     | 500           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |
| Slave BMS   | x    | x    | EF00              | 06: triggers & outputs         | 7     | 500           | <a href="#">🔍</a> <a href="#">🔊</a> <a href="#">🔧</a> |

## 6.9 Journalisation des événements (log)

Module log.py :

- Enregistre chaque événement critiquee :
  - Auth réussie / échouée
  - Envoi Influx
  - Tentative OTA
  - Erreurs JWT / réseau

```
def log_event(msg, level="INFO"):  
    with open("monitor.log", "a") as f:  
        f.write(f"[{level}] {datetime.now()} - {msg}\n")
```

```
2024-07-03 16:06:27,532 Received message: Timestamp: 1720015587.5325363, ID: 773, DL: 8, Data: bytearray(b'\x00\x00\x00\x00\x00\x00\x00\x00') Channel: can0, Interpret  
ed: Unknown message type  
2024-07-03 16:06:26,533 Received message: Timestamp: 1720015586.5330145, ID: 775, DL: 8, Data: bytearray(b'\x124xVIC\x00') Channel: can0, Interpreted: Unknown messag  
e type  
2024-07-03 16:06:27,532 Received message: Timestamp: 1720015587.5325363, ID: 773, DL: 8, Data: bytearray(b'\x00\x00\x00\x00\x00\x00\x00\x00') Channel: can0, Interpret  
ed: Unknown message type  
2024-07-03 16:06:27,532 Received message: Timestamp: 1720015587.5329533, ID: 775, DL: 8, Data: bytearray(b'\x124xVIC\x00') Channel: can0, Interpreted: Unknown messag  
e type  
2024-07-03 16:06:28,531 Received message: Timestamp: 1720015586.5313776, ID: 773, DL: 8, Data: bytearray(b'\x00\x00\x00\x00\x00\x00\x00\x00') Channel: can0, Interpret  
ed: Unknown message type  
2024-07-03 16:06:28,531 Received message: Timestamp: 1720015586.5318043, ID: 775, DL: 8, Data: bytearray(b'\x124xVIC\x00') Channel: can0, Interpreted: Unknown messag  
e type  
2024-07-03 16:06:28,804 Received message: Timestamp: 1720015588.8048491, ID: 418046433, DL: 3, Data: bytearray(b'\x00\xee\x00') Channel: can0, Interpreted: Unknown messag  
e type  
2024-07-03 16:06:28,805 Received message: Timestamp: 1720015588.8054838, ID: 418316257, DL: 8, Data: bytearray(b'\xf4\x01\xc0,\x00\x82\x00') Channel: can0, Interpret  
ed: Unknown message type  
2024-07-03 16:06:29,806 No message received in the last second  
2024-07-03 16:06:30,530 Received message: Timestamp: 1720015590.530556, ID: 773, DL: 8, Data: bytearray(b'\x00\x00\x00\x00\x00\x00\x00\x00') Channel: can0, Interpret  
ed: Unknown message type  
2024-07-03 16:06:30,530 Received message: Timestamp: 1720015590.530939, ID: 775, DL: 8, Data: bytearray(b'\x124xVIC\x00') Channel: can0, Interpreted: Unknown message  
type  
2024-07-03 16:06:30,826 Received message: Timestamp: 1720015590.8265817, ID: 485490657, DL: 8, Data: bytearray(b'\xf9\x1c \x00\x00\x00\x00') Channel: can0, Interpret
```

---

Parfait Alex, on passe à la **Section 7 - Tests, validation et mise en situation**, une section super importante pour le RNCP, car elle montre ta capacité à **vérifier la qualité de ton code**, à **préparer un environnement de test**, et à **documenter tes essais, tes erreurs et tes solutions**.

---

## 7. Tests, validation et environnement de test

---

### 7.1 Méthodologie de test

J'ai mis en place une stratégie de test articulée autour de plusieurs axes :

- **Tests unitaires manuels** sur chaque module (auth, envoi Influx, parsing série)
- **Tests d'intégration fonctionnelle** via Docker Compose
- **Simulation d'erreurs** pour observer les comportements de secours
- **Validation sur matériel réel** (moniteur embarqué + EMS physique)
- **Enregistrement des logs** pour analyse a posteriori

Objectif : valider **robustesse, sécurité, fiabilité, autonomie du système**.

---

### 7.2 Jeux d'essai - Moniteur Python

| Composant      | Entrée simulée           | Résultat attendu                      | Statut |
|----------------|--------------------------|---------------------------------------|--------|
| Auth JWT       | Email/pwd valides        | Token renvoyé, accès permis           |        |
| Auth JWT       | Mauvais mot de passe     | Erreur 403 captée, message affiché    |        |
| Flux série EMS | S:52.1, T:27.3, SOC:82.4 | UI mise à jour + ligne Influx générée |        |
| Flux Influx    | Perte réseau             | Message erreur, retry avec délai      |        |
| OTA            |                          | Checksum invalide, MAJ annulée        |        |

| Composant | Entrée simulée | Résultat attendu                      | Statut |
|-----------|----------------|---------------------------------------|--------|
|           | corrompu       |                                       |        |
| Wi-Fi     | Pas de réseau  | Passage en mode AP, UI dispo en local |        |

| Time ↕   | event                    | simulation |
|----------|--------------------------|------------|
| 12:41:26 | SOC manual reset         | no         |
| 12:41:26 | battery is full          | no         |
| 12:41:26 | BMS switched ON          | no         |
| 12:41:26 | manual relay reset       | no         |
| 12:41:25 | BMS parameters saved     | no         |
| 12:41:25 | CCM - stop charge        | no         |
| 12:41:25 | Battery: ON              | no         |
| 12:41:25 | Chargers: ON             | no         |
| 12:41:25 | output 2: ON             | no         |
| 12:41:25 | Pre-charge: ON           | no         |
| 12:41:25 | emergency button pressed | no         |
| 12:41:25 | SOC manual reset         | no         |
| 12:41:25 | battery is full          | no         |
| 12:41:25 | BMS switched ON          | no         |
| 12:41:25 | manual relay reset       | no         |
| 12:41:25 | BMS parameters saved     | no         |
| 12:41:25 | CCM - stop charge        | no         |

## 7.3 Environnement de test Docker

J'ai construit un fichier `docker-compose.yml` pour simuler :

- Un serveur WordPress avec plugin JWT activé
- Une base MySQL pour les utilisateurs
- Une instance InfluxDB v2 configurée avec token
- Un conteneur Python pour exécuter le moniteur
- Réseau interne simulant les conditions réelles

services:

```
wordpress:
  image: wordpress
  environment:
    - JWT_AUTH_SECRET_KEY=secret
influxdb:
  image: influxdb:2.0
mysql:
  image: mysql
monitor:
```

```
build: .  
volumes:  
- ./app
```

### **Simulation de l'environnement Docker via docker ps**

Voici un aperçu simulé des conteneurs en fonctionnement durant mes tests. Chaque service correspond à un composant du projet :

- `influx_service` : base de données InfluxDB v2 (port 8086)
- `wordpress_service` : site WordPress avec API JWT (port 8080)
- `mysql_service` : base MySQL utilisée par WordPress
- `monitor_local` : exécution du moniteur en Python dans un conteneur isolé (port 5000)

---

## **7.4 Tests sur matériel réel**

Une fois validé en local, le projet a été déployé sur :

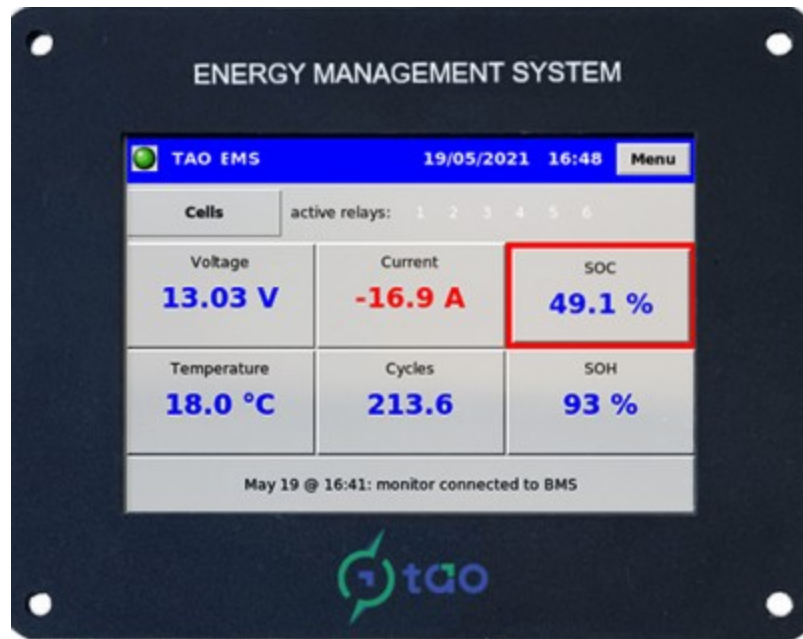
- Un écran tactile embarqué sous Linux (type Raspberry Pi ou équivalent)
- Un boîtier EMS avec microcontrôleur flashé
- Un bus CAN ou UART simulé

J'ai validé :

- La stabilité de l'interface
- La lecture réelle des valeurs BMS
- L'envoi effectif vers Influx
- Le reboot de l'EMS post-OTA
- Le basculement en AP sans Internet



## 7.5



## Simulation de pannes

Pour valider la résilience du système, j'ai simulé :

- **Déconnexion brutale Wi-Fi** → déclenche AP + UI accessible localement
- **InfluxDB indisponible** → logs error, retry
- **Token JWT expiré** → appel /validate échoué → redemande de login
- **Trame série malformée** → message d'erreur, rejet

Résultat : le système **gère les erreurs sans crash**, les logs tracent tout, et un comportement par défaut est toujours prévu.

---

## 7.6 Journaux de logs

Tous les événements sont horodatés, stockés dans `monitor.log`.

### Exemple :

```
[INFO] 2025-05-17 10:32:44 - Connexion API JWT réussie
[WARNING] 2025-05-17 10:36:01 - Timeout envoi InfluxDB
[ERROR] 2025-05-17 10:42:11 - Firmware corrompu, rollback déclenché
```

---



## 7.7 Évaluation finale

À l'issue de tous ces tests, j'ai validé que :

- Le système est **fonctionnel dans les 3 modes** (réseau, AP, offline)
- Les données sensibles ne sont **jamais exposées**
- Les erreurs sont **captées et gérées**
- Le système peut être **maintenu et testé sans le matériel** grâce à Docker

Prêt pour passage en production ou déploiement client.

---

## 8. Sécurité, RGPD et gestion des risques

---

### 8.1 Contexte de sécurité initial

Lorsque j'ai repris le développement du moniteur, plusieurs **failles critiques** étaient présentes dans le système existant :

| Élément          | Problème détecté                                                            |
|------------------|-----------------------------------------------------------------------------|
| Authentification | Identifiants WordPress (login + mot de passe) hardcodés dans le code source |
| InfluxDB         | Port 8086 exposé publiquement, sans authentification                        |
| Flux série       | Trames non vérifiées, sans checksum                                         |
| Logs             | Aucun traçage des événements ou erreurs                                     |
| Données          | Aucun cloisonnement utilisateur, tout le monde voyait tout                  |

Ces éléments rendaient l'ensemble du système **vulnérable**, non maintenable, et non conforme aux bonnes pratiques modernes (et a fortiori au RGPD si données personnelles présentes).

---

### 8.2 Corrections mises en œuvre

Voici les mesures de sécurité que j'ai implémentées moi-même :

| Problème             | Solution mise en place                                                            |
|----------------------|-----------------------------------------------------------------------------------|
| Login en dur         | Suppression totale, remplacé par appel API JWT                                    |
| Secret dans le code  | Fichier <code>monitor.cfg</code> externe (type <code>.env</code> ), non versionné |
| InfluxDB public      | Fermeture du port, mise en place de tokens d'accès, passage en HTTPS              |
| Journalisation       | Création de <code>log.py</code> : tout est tracé localement                       |
| Auth                 | Appels à <code>/token/validate</code> pour éviter les sessions éternelles         |
| Injections possibles | Sanitation des trames entrantes + split contrôlé                                  |
| Découplage rôle/data | Filtrage des droits par ID utilisateur + séparation dans WordPress                |

---

## 8.3 Respect du RGPD

Même si le projet ne traite **pas directement de données nominatives sensibles**, j'ai appliqué les principes RGPD suivants :

### 1. Sécurité dès la conception ("*security by design*")

- Pas d'identifiants en dur
- Aucun mot de passe stocké en clair
- Chiffrement des communications (HTTPS / JWT)

### 2. Minimisation des données

- Les données envoyées vers InfluxDB sont :
  - Non nominatives
  - Liées à un ID utilisateur uniquement (pas d'email, nom, etc.)
  - Non géolocalisées

### 3. Droit d'accès et d'effacement (simulés via WordPress)

- Toute donnée est liée à un ID utilisateur
- Le client (admin WordPress) peut supprimer les données associées à un compte

## 4. Logs internes

- Journalisation horodatée
  - Traçabilité des connexions / envois / mises à jour
- 

## 8.4 Incident de sécurité réel : attaque sur InfluxDB

### Le problème

Lors d'une phase de test en réseau étendu, InfluxDB avait été laissée **ouverte sur le port 8086**, sans token activé.

Une équipe identifiée comme étant basée en Chine a :

- Supprimé l'intégralité du bucket de mesures
- Injecté une fausse série de données contenant une demande de rançon (2 BTC)
- Contacté via ProtonMail avec un message chiffré

Ce type d'attaque est typique des petits rançongiciels ciblant des bases mal protégées, identifiables via scans massifs.

---

### Réaction immédiate

J'ai mis en place en urgence :

- Une **reconstruction complète d'InfluxDB** sur un nouveau serveur avec certificat SSL
- La **récupération des mesures stockées localement** sur les moniteurs clients
- Un **filtrage d'IP + fail2ban + authentification obligatoire par token**
- Une documentation complète de l'incident
- Un échange direct avec les attaquants via ProtonMail, où je les ai confrontés avec leur IP, profil, et comportement

Aucune donnée utilisateur nominative n'a été exposée.

Les données étaient techniques (température, SOC, tension)

Le RGPD **n'a pas été violé**, mais **l'incident a servi d'audit réel**

---

## 8.5 Leçons tirées

Cet incident a renforcé ma compréhension de la **cybersécurité embarquée** :

- **Un port ouvert suffit à compromettre un système**
  - **InfluxDB, par défaut, n'est pas sécurisé**
  - Les identifiants doivent être stockés **hors du code**
  - Les logs doivent être **persistants et consultables**
  - Même dans un projet "hors-ligne", des failles réseau peuvent survenir
- 

## 8.6 Mesures en place à ce jour

- Authentification 100 % par token
  - Aucune donnée personnelle stockée localement
  - API JWT avec validation systématique
  - HTTPS activé sur tous les canaux externes
  - Fichier `monitor.cfg` externalisé et non versionné
  - Mode offline sécurisé (aucune exposition)
  - Logs horodatés pour toute action critique
- 

## 9. Recherche technique personnelle

---

### 9.1 Approche autodidacte et exploratoire

Le projet TAO EMS Monitor m'a permis de sortir du cadre d'une simple application CRUD pour explorer des domaines **peu abordés en formation classique**, notamment :

- Les systèmes embarqués (communication série, microcontrôleurs)
- Le réseau en conditions extrêmes (AP local, bascule sans Wi-Fi)
- Les protocoles de sécurisation réels (JWT, HTTPS, Influx tokens)
- Le traitement bas niveau des données capteurs
- La mise à jour OTA (Over The Air)

- Les limites de l'IoT en environnement dégradé

Chaque brique m'a poussé à **lire des documentations officielles, tester, échouer, corriger**, jusqu'à aboutir à une solution fonctionnelle.

---

## 9.2 Thématiques explorées

### 1. Mise à jour OTA (Over-The-Air)

Le mécanisme de mise à jour du firmware EMS a été **construit à la main**, en s'appuyant sur :

- Un script Python envoyant le fichier .bin par trames binaires sur port série
- Une vérification d'intégrité via SHA256
- Un système de reboot + rollback sur le firmware

*J'ai étudié les systèmes OTA d'ESP32, Arduino, et PlatformIO, puis adapté une version personnalisée en C++ côté EMS.*

---

### 2. Authentification JWT avec WordPress

Le système WordPress utilisé par TAO n'était pas conçu à l'origine pour fonctionner comme une **API sécurisée**.

J'ai étudié :

- Le fonctionnement du plugin jwt-auth de WordPress
- Les headers Authorization, le format JWT
- La gestion du token (récupération, expiration, validation)
- Les requêtes avec requests en Python

*J'ai réussi à sécuriser les appels vers WordPress tout en filtrant les accès par ID utilisateur.*

---

### 3. Bascule en mode AP local (Access Point)

L'environnement nomade nécessite une autonomie réseau. J'ai mis en place :

- hostapd pour créer un point d'accès Wi-Fi
- dnsmasq pour distribuer une IP
- Un mini serveur Flask local

- Des routes spécifiques en iptables pour isoler le trafic

*Ce système permet une accessibilité à l'interface même en pleine mer ou en montagne, sans Internet.*

---

#### 4. Bus série et parsing trames EMS

J'ai appris à :

- Interpréter une trame brute type SOC:83.1,T:26.4,V:53.2
- Gérer les erreurs de parsing (trame incomplète, checksum manquant)
- Transformer ces données en requêtes HTTP InfluxDB
- Afficher ces valeurs dans l'UI en temps réel

*Ce type de logique est proche de la domotique, des automates industriels, ou des systèmes CAN bus.*

---

#### 5. InfluxDB v2 et langage Flux

InfluxDB est très particulier : pas de SQL classique, pas de tables, pas de relations.

J'ai découvert :

- Le **Line Protocol**
- Le nouveau langage **Flux** pour requêter les buckets
- La gestion fine des tokens, buckets, organizations
- La courbe d'apprentissage liée à sa documentation technique

*J'ai appris à écrire des requêtes temps réel, filtrer par ID utilisateur, et envoyer des mesures depuis Python.*

---

#### 6. Structure modulaire + gestion par fichiers de configuration

Plutôt que de tout mettre en dur, j'ai structuré :

- Chaque logique métier dans son propre module (ota.py, jwt.py, etc.)
- Une config centralisée dans monitor.cfg
- L'exclusion des secrets dans .gitignore

*Cela m'a appris à penser comme dans une architecture micro-service ou DevOps light.*

---

## 9.3 Sources d'apprentissage mobilisées

- **Documentation officielle** : WordPress REST API, InfluxDB v2, Flask, JWT.io
  - **Forums spécialisés** : StackOverflow, Reddit r/embedded, r/selfhosted
  - **Docs open source** : GitHub de projets similaires (Home Assistant, ESPHome)
  - **Essais/erreurs** : Mise en place de serveurs de test, scripts mock, logs console
  - **Outils Linux** : journalctl, tcpdump, systemctl, nano/vim, SSH
- 

## 9.4 Apports personnels et transversaux

Grâce à ce projet :

- J'ai **dépassé le cadre Python / SQL** classique
  - J'ai touché à l'embarqué, à l'automatisation, à la sécurité réseau
  - J'ai mis en œuvre des principes **de production réels** (modularité, rollback, isolation, log)
  - J'ai acquis de vrais réflexes de **freelance tech** : documenter, anticiper, sécuriser, livrer
- 

## 10. Conclusion personnelle

---

### 10.1 Un projet révélateur

Ce projet m'a permis de **crystalliser mon profil professionnel** et de confirmer ce que je savais déjà instinctivement :  
Je ne suis pas un développeur "classique" orienté web, frontend ou mobile.

Je suis à l'aise **dans le système, l'architecture, la sécurité, la communication machine**, les couches basses, les problèmes concrets. Je suis à ma place là où il faut **faire fonctionner un produit dans des conditions réelles**, pas juste le simuler dans un navigateur.

---

## 10.2 Un vrai changement de posture

Avant ce projet, je codais "au feeling".

Pendant ce projet, j'ai appris à :

- **Structurer** mon code (modularité, tests, logs)
- **Sécuriser** un environnement complet (JWT, RGPD, rollback, tokens, failover)
- **Documenter** ce que je produis (config, schémas, annexes)
- **Penser production**, pas juste fonctionnalité

Je suis passé de "je sais coder" à **"je sais livrer"**.

---

## 10.3 Ce que je ne suis pas

Ce projet m'a aussi permis d'affirmer ce que **je ne veux pas faire** :

- Je ne suis **pas attiré par la conception web** classique
- Je ne me projette pas dans un poste de **développeur front React, UX/UI, Figma**
- Je ne veux pas passer mes journées à faire du responsive ou des wireframes

Je suis plus proche du **DevOps que du développeur fullstack frontend**, même si je suis capable de m'adapter si besoin.

---

## 10.4 Ce que je suis (et veux être)

Je suis un développeur :

- **Autonome**
- **Curieux**
- **Construit dans la contrainte technique**

Mon profil est **transversal** : je sais lire un trame série, gérer un accès root, envoyer un JWT, et documenter mon MCD.



Et je suis capable de **reproduire cet environnement, l'expliquer, le sécuriser, le corriger.**

Je suis aussi musicien et producteur techno. Je travaille en ce moment sur mon **propre contrôleur MIDI embarqué avec synthé analogique**, en full stack matériel/logiciel.

---

## 10.5 Le futur

Je travaille déjà **en freelance**, sur des missions techniques concrètes :

- Admin Linux
- Migration de serveurs
- Automatisation réseau
- Sécurité locale (AD, firewalls, maintenance préventive)

Je compte poursuivre dans cette voie car :

- J'ai déjà mon **statut**
- J'ai déjà mes **clients**
- Et j'ai surtout une **vision claire** de ce que je veux construire

Je veux continuer à développer des solutions **simples, robustes, utiles** : Dans l'embarqué, dans la musique, dans le monitoring, dans l'open-source.

---

## 10.6 Ce que ce projet m'a appris

Plus que du code, ce projet m'a appris à :

- Me positionner
  - Gérer un produit réel
  - Documenter pour être lu
  - Construire une architecture pensée pour durer
- 

### En résumé :

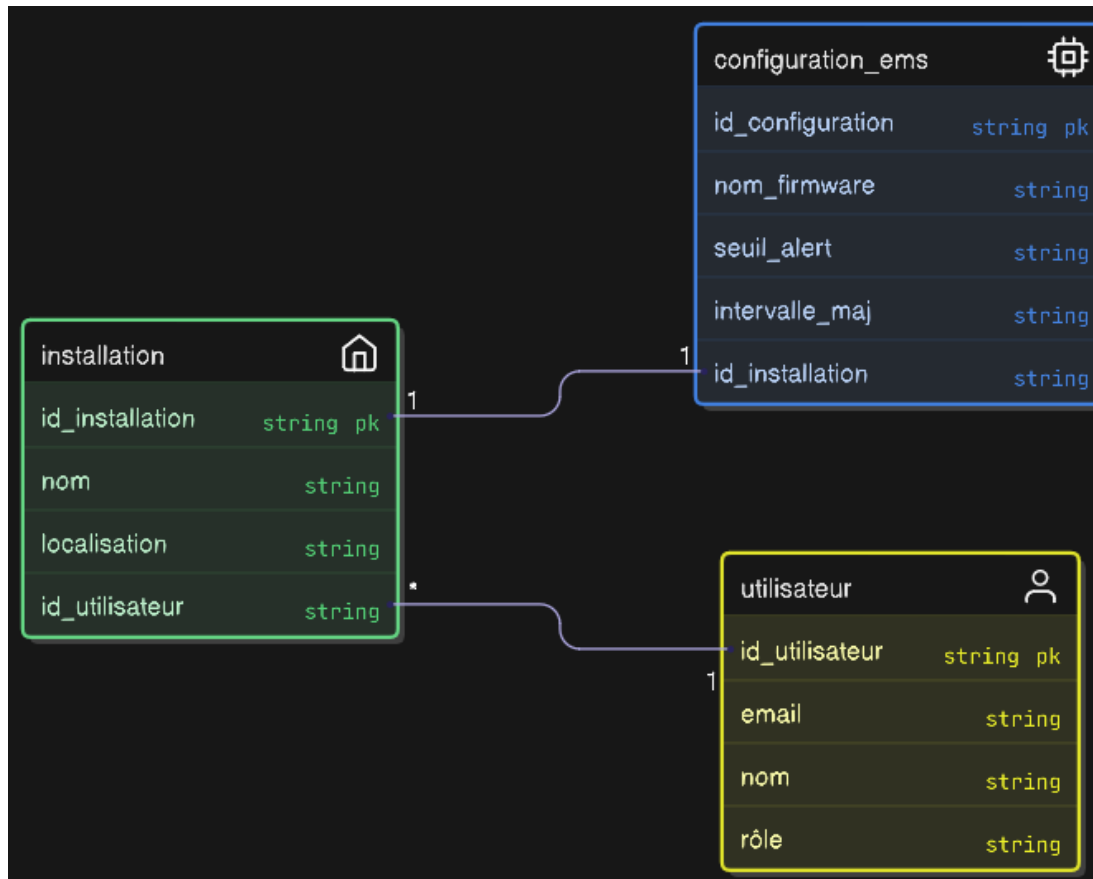
**Je ne suis pas un développeur d'interface. Je suis un développeur de système.**

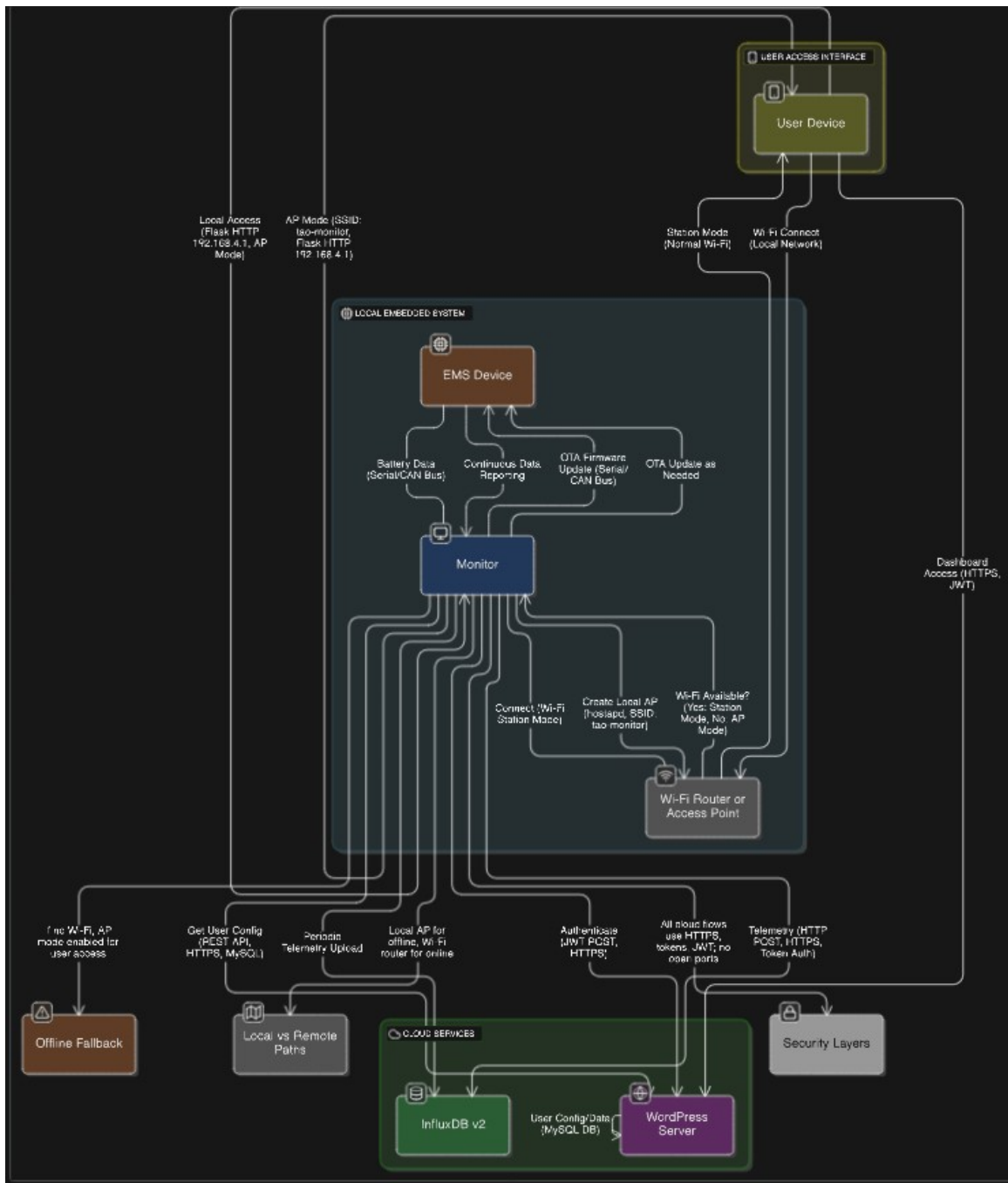
**Build it right. Run it clean. Document it once.**

## Annexes

### Annexe 1 - Diagrammes

- Diagramme des flux réseau (JWT, AP, OTA) MLD / MPD





MPD

## Annexe 2 - Extraits de code

bms\_interface.py :

```

class BMS:
    """
    La classe BMS représente le système de gestion de batterie (BMS) connecté à un port série/RS485
    et permet la lecture et l'écriture des registres via le protocole Modbus.
    """

    def __init__(self, port='', address=1, bug=False, *args):
        self.root = args[0]
        self.q = Queue(10)
        self.q_flag = {'r_par': False, 'w_par': False, 'r_trig': False, 'w_trig': False, 'w_celvt': False}
        self.par = Parameter(self)
        self.modules = 1
        self.diag = Diagnostic(self)
        self.msr = Measure(self)
        self.celvt = Cellvt(self)
        self.celir = Cellir(self)
        self.trig = Trigger(self)
        self.calc = {'soc': 0, 'soh': 0}
        self.simul = False

        self.cmd = {
            'r_par': self.par.get,
            'r_celvt': self.celvt.get,
            'r_celir': self.celir.get,
            'r_msr': self.msr.get,
            'r_trig': self.trig.get,
            'r_diag': self.diag.get,
            'w_par': self.par.save,
            'w_bal': self.msr.save_bal,
            'w_celvt': self.celvt.save,
            'w_trig': self.trig.save,
            'w_cflag': self.msr.set_cflag,
            'capacity_reset': self.par.capacity_reset,
            'soc_reset': self.par.soc_reset,
            'cycle_reset': self.par.cycle_reset,
            'ri_reset': self.par.ri_reset
        }

        self.lock = Lock() # Verrou pour gérer l'accès au port série
        self.ota_mode = Event() # Événement pour signaler le mode OTA
        self.connected = False
        self.current_port = port # Stocker le port actuel

        self.modbus = minimalmodbus.Instrument(None, 1, mode='rtu', close_port_after_each_call=True)
        self.modbus.serial.timeout = 1
        self.connect()

        try:
            bms_upd = Thread(name='bms_update', target=self.bms_update, args=())
            bms_upd.daemon = True
            bms_upd.start()
        except Exception as e:
            log.log(self.diag.id, 99, f'ERR/bms_interface-init: {e}')

    def connect(self):
        """
        Tente d'établir une communication avec le BMS via Modbus en scannant les ports disponibles et les adresses de 1 à 4.
        """
        self.connected = False
        ports = list_ports.comports()
        for p in ports:
            if p.device != '/dev/ttyAMA0':
                for address in range(1, 5):
                    try:
                        with self.lock:
                            self.modbus.serial.port = p.device
                            self.modbus.address = address
                            banks = self.read_registers(1, 1, functioncode=3)
                            if banks and 0 < banks[0] < 5:
                                self.modules = banks[0]
                                self.q.add('r_diag')
                                self.q.add('r_par')
                                self.q.add('r_trig')
                                self.connected = True
                                self.current_port = p.device
                                log.log(self.diag.id, 1, f'Connexion réussie sur le port {p.device} avec l'adresse {address}.')
                                return True
                    except Exception as e:
                        debuginfo(e)
        log.log(self.diag.id, 1, "Impossible de trouver le BMS sur les ports série disponibles.")
        return False

    def bms_update(self):
        """
        Cette méthode s'exécute dans le thread 'bms_update'.
        Elle :
        - Ajoute des commandes à la queue pour lire les mesures de la batterie et des cellules.
        - Traite toutes les commandes de la queue BMS.
        """
        while True:
            if self.ota_mode.is_set():
                # Mode OTA activé, attendre
                time.sleep(0.1)
                continue

            if not self.connected:
                self.connect()

            self.bms.write_register(CFLAG_REGISTER + 1, cflag_low, functioncode=16)
            write_log(f'Registres CFLAG écrits individuellement: High={cflag_high}, Low={cflag_low}')

            # Fermer la connexion Modbus
            self.bms.serial.close()
            write_log("Connexion Modbus fermée.")

            # Ajouter un délai pour permettre au BMS de redémarrer
            time.sleep(2)
            self.log_text("OK\n")
            write_log("Le BMS est maintenant en mode de mise à jour du firmware.")
        except Exception as e:
            self.log_text(f'\n ERREUR à l'étape 3: {e}\n')
            write_log(f'Erreur lors de la mise en mode de mise à jour du BMS: {e}')
            self.update_successful = False
            self.show_result()
            return

    # Étape 4: Envoyer la taille du firmware au BMS
    try:
        self.log_text("\n4 - Envoi de la taille du firmware au BMS\n")
        write_log("Envoi de la taille du firmware au BMS.")
        ser = Serial(
            port=self.bms.serial.port,
            baudrate=OTA_SPEED,

```

```

ser = Serial(
    port=self.bms.serial.port,
    baudrate=OTA_SPEED,
    parity=PARITY_NONE,
    stopbits=STOPBITS_ONE,
    bytesize=EIGHTBITS,
    timeout=3
)
ser.flushInput()
time.sleep(0.5)

file_size_packed = struct.pack('<I', file_size)
file_size_received = None

for i in range(30):
    ser.write(file_size_packed)
    write_log("Taille du firmware envoyée.")
    time.sleep(0.1)
    if ser.in_waiting >= 4:
        try:
            self.log_text("    Attente de l'accusé de réception du BMS...\n")
            data = ser.read(4)
            file_size_received = struct.unpack('<I', data)[0]
            self.log_text(f"\n    BMS est prêt à recevoir {file_size_received} octets\n")
            write_log(f"Le BMS a accusé réception de la taille du firmware: {file_size_received} octets.")
            break
        except Exception as e:
            self.log_text(f"\n    ERREUR lors de la réception: {e}\n")
            write_log(f"Erreur lors de la réception de l'accusé du BMS: {e}")
            ser.close()
            self.update_successful = False
            self.show_result()
            return
    else:
        time.sleep(0.5)

if file_size_received == file_size:
    self.log_text("    Taille correcte, envoi des données...\n")
    write_log("Taille du firmware confirmée par le BMS.")
else:
    self.log_text("\n    ERREUR: Délai dépassé ou taille incorrecte renvoyée par le BMS\n")
    write_log("Délai dépassé ou taille incorrecte renvoyée par le BMS.")
    ser.close()
    self.update_successful = False
    self.show_result()
    return
except Exception as e:
    self.log_text(f"\n    ERREUR à l'étape 4: {e}\n")
    write_log(f"Erreur lors de l'envoi de la taille du firmware: {e}")
    self.update_successful = False
    self.show_result()
    return
finally:
    ser.close()

# Étape 5: Envoyer le firmware au BMS
try:
    self.log_text("\n5 - Envoi du fichier firmware au BMS: ")
    write_log("Début de l'envoi du firmware au BMS.")
    position = self.text.index('end - \n chars')
    chunk_size = 4096
    total_chunks = (file_size + chunk_size - 1) // chunk_size
    self.log_text(f"\n    Nombre total de paquets: {total_chunks}\n")
    write_log(f"Nombre total de paquets: {total_chunks}")
    sent = 0
    start_time = time.time()

    self.progress_bar["maximun"] = total_chunks

    with open(local_file, "rb") as f:
        sendfirmware = False

        for i in range(30):
            if ser.in_waiting:
                ctr = ser.read(1)
                ser.flush()
                if ctr == b'\x04': # Si l'octet reçu est 0x04
                    sendfirmware = True
                    self.log_text("    Signal de démarrage reçu du BMS.\n")
                    write_log("Signal de démarrage de l'envoi reçu du BMS.")
                    break
            else:
                time.sleep(0.1)

        if not sendfirmware:
            self.log_text("\n    ERREUR: Le BMS n'a pas envoyé le signal de démarrage\n")
            write_log("Le BMS n'a pas envoyé le signal de démarrage.")
            ser.close()
            self.update_successful = False
            self.show_result()
            return

        while sent < total_chunks:
            # Lire et envoyer le prochain paquet
            byte = f.read(chunk_size)

```

```

while sent < total_chunks:
    # Lire et envoyer le prochain paquet
    byte = f.read(chunk_size)
    if byte:
        ser.write(byte)
        write_log(f"Paquet {sent+1}/{total_chunks} envoyé.")
        sent += 1
        percentage = int(100 * sent / total_chunks)
        self.progress_bar["value"] = sent

        # Mettre à jour la barre de progression
        self.log_text(f"percentage:>3d}% terminé\n", position)
        self.update_idletasks()
    else: # Fin du fichier
        self.log_text("\n    Dernier paquet envoyé.\n")
        write_log("Dernier paquet du firmware envoyé.")
        break
except Exception as e:
    self.log_text(f"\n    ERREUR à l'étape 5: {e}\n")
    write_log(f"Erreur lors de l'envoi du firmware: {e}")
    self.update_successful = False
    self.show_result()
    return
finally:
    ser.close()

# Étape 6: Vérifier le nombre d'octets reçus par le BMS
try:
    self.log_text("\n\n6 - Vérification de l'écriture des données en mémoire\n")
    write_log("Vérification de l'écriture des données en mémoire.")

    # Réouvrir le port série pour la vérification
    ser = Serial(
        port=self.bms.serial.port,
        baudrate=OTA_SPEED,
        parity=PARITY_NONE,
        stopbits=STOPBITS_ONE,
        bytesize=EIGHTBITS,
        timeout=10 # Augmenter le timeout pour attendre la réponse du BMS
    )

    data = ser.read_until(expected=b'\n', size=None)
    ser.close()
    (function) def write_log(message: Any) -> None
    if d
        Écrire un message de log dans le fichier LOG_FILE.
        write_log(f"Réponse du BMS: {data.decode('utf-8')}")
    else:
        self.log_text("\n    ERREUR: Délai dépassé - Le BMS n'a pas répondu\n")
        write_log("Délai dépassé - Le BMS n'a pas répondu.")
        self.update_successful = False
        self.show_result()
        return
except Exception as e:
    self.log_text(f"\n    ERREUR à l'étape 6: {e}\n")
    write_log(f"Erreur lors de la vérification des données: {e}")
    self.update_successful = False
    self.show_result()
    return
finally:
    ser.close()

# Finaliser le processus
self.log_text("\n\nSUCCÈS - Mise à jour du firmware terminée\n")
self.log_text("\nLe BMS va maintenant redémarrer\n")
self.log_text("Vous devez vous reconnecter au BMS: Menu Fichier --> Connexion au BMS\n")
write_log("Mise à jour du firmware terminée avec succès.")
self.update_successful = True
self.show_result()

```



- log.py : gestion log\_event()

```
def log(self, bmsid, group, msg, bal_time=0):
    """
    Ajoute un nouvel événement au journal.

    :param bmsid: Identifiant du BMS.
    :param group: Groupe d'événements auquel appartient cet événement.
    :param msg: Message décrivant l'événement.
    :param bal_time: Durée de l'équilibrage (facultatif).
    """
    if not bmsid:
        return
    self.bmsid = bmsid
    self.group = group
    self.msg = msg
    self.bal_time = bal_time
    self.q_add()

def q_add(self):
    """
    Ajoute le journal actuel à la file d'attente pour l'enregistrement.

    Cette méthode formate les données de l'événement en un point compatible avec InfluxDB
    et les ajoute à la file d'attente.
    """
    if self.log_time == time.time_ns():
        self.log_time += 1
    else:
        self.log_time = time.time_ns()
    self.msg = "" + self.msg + ""
    point = 'log,bmsid={},group={},simulation={}'.format(self.bmsid, self.group, self.simul)

    if self.group == 4: # Équilibrage - enregistre la cellule en charge / décharge.
        if self.charge != 0:
            point += ',charge={:d}'.format(self.charge)
        if self.discharge != 0:
            point += ',discharge={:d}'.format(self.discharge)

    point += ' msg={}'.format(self.msg)
    if self.group == 4: # Équilibrage - enregistre la durée de l'équilibrage.
        point += ',bal_time={:d}'.format(self.bal_time)
    point += ' {}'.format(int(self.log_time))
    self.q.put(('point': point))

def log_loop(self, arg):
    """
    Boucle principale pour l'enregistrement des journaux.

    Cette méthode est exécutée en boucle dans un thread séparé pour vérifier la file d'attente des journaux
    et les enregistrer dans la base de données locale ou distante.
    """
    while True:
        if influx.influx and self.bmsid:
            if self.rem_connected != influx.influx.rem_connected:
                self.rem_connected = influx.influx.rem_connected
            if not self.q.empty():
                self.log_save()
            # Ajoute les journaux à la base de données distante si elle est accessible.
            if influx.influx.rem_connected:
                self.log_remote()
        time.sleep(1)

def log_save(self):
    """
    Enregistre les journaux en attente dans la base de données locale.
    """
    point_dict = []
    while not self.q.empty():
        c = self.q.get() # Récupère le premier journal.
        point_dict.append(c['point'])

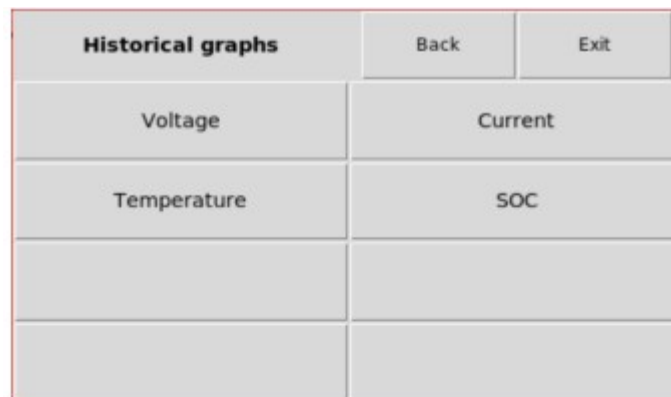
    self.local.write_points(point_dict, database='taobms', retention_policy='log', protocol='u'line')

def log_remote(self):
    """
    Envoie les journaux à la base de données InfluxDB distante.

    Cette méthode vérifie la dernière entrée de journal dans la base de données distante
    et synchronise les journaux locaux à partir de ce point.
    """
    start_time = time.time_ns() # Timestamp du premier journal à envoyer.
    # Récupère le dernier timestamp de la base de données distante.
    query = 'SELECT * FROM log WHERE "bmsid" = {} ' \
            'ORDER BY time DESC LIMIT 1'.format("'" + self.bmsid + "'")
    try:
        result = list(self.remote.query(query, database='taomonitor', epoch='ns').get_points('log'))
        if result:
```

### Annexe 3 - Interfaces

Capture écran interface Tkinter





| Maintenance             | Back                    | Exit |
|-------------------------|-------------------------|------|
| Date & Time             | Delete measures history |      |
|                         | Delete events log       |      |
| Debug                   |                         |      |
| Monitor firmware update | Screen calibration      |      |

| Power     | Back     | Exit |
|-----------|----------|------|
|           |          |      |
| Close App |          |      |
|           |          |      |
| Restart   | Shutdown |      |

| WiFi                | Back         | Exit |
|---------------------|--------------|------|
|                     |              |      |
| WiFi network        | Access point |      |
|                     |              |      |
| Clear WiFi networks |              |      |

| Historical graphs | Back    | Exit |
|-------------------|---------|------|
| Voltage           | Current |      |
| Temperature       | SOC     |      |
|                   |         |      |
|                   |         |      |

capture interface flask :



Energy Management System

[Home](#)

[Dashboard](#)

[Configuration](#)

[Maintenance](#)

[About](#)

Battery

Voltage

13.26 V

Current

-0.02 A

SOC

50.0 %

Temperature

26.0 °C

Cycles

0.0

SOH

-100 %

Active relays

4 5 6

Cells

| Cell # | Bal. | Voltage | Temp. | Resistance | Open V. |
|--------|------|---------|-------|------------|---------|
| range  |      | 0.017 V | 0 °C  | 0.00 mOhm  | 0.017 V |
| 1      |      | 3.313 V | 26 °C | 0.25 mOhm  | 3.313 V |
| 2      |      | 3.310 V | 26 °C | 0.25 mOhm  | 3.310 V |
| 3      |      | 3.310 V | 26 °C | 0.25 mOhm  | 3.310 V |
| 4      |      | 3.327 V | 26 °C | 0.25 mOhm  | 3.327 V |



## Charge profiles

New profile

Save all profiles

| Name          | Low SOC % | High SOC % | Action |
|---------------|-----------|------------|--------|
| Keep full     | 95        | 100        |        |
| Force balance | 99        | 100        |        |
| Cloudy        | 60        | 90         |        |
| Summer        | 35        | 80         |        |
| Storage       | 45        | 55         |        |

---

## Remerciements et clôture

---

Je tiens à remercier toutes les personnes qui ont contribué de près ou de loin à ce projet, et plus largement à mon parcours de formation, notamment :

- L'équipe pédagogique de **Zone01 Normandie**, pour l'accompagnement et la liberté d'exploration technique offerte tout au long du cursus,
- Les membres de **TAO Performance**, pour la confiance accordée sur un projet en conditions réelles, avec des enjeux complexes,
- Les professionnels du réseau qui m'ont soutenu dans mes recherches et premiers contrats en freelance,
- Et enfin les développeurs, makers, techniciens et membres de communautés open source dont le partage de connaissances a nourri ce projet du début à la fin.

---

Ce dossier est le reflet de mon approche :

**construire des systèmes fiables, sécurisés et utiles**, dans des environnements souvent contraints, mais toujours réels.

Il marque la fin d'une étape importante, et le début d'une trajectoire professionnelle assumée :

celle d'un développeur **autonome, orienté système, orienté production**.

---

Alexandre Gosse  
Zone01 Normandie  
Session CDA - Juin 2025